PREDICTIVE MAINTENANCE OF APS FAILURES IN SCANIA TRUCKS: A MACHINE LEARNING APPROACH USING STACKING ENSEMBLES

A PREPRINT

BISWARUP MUKHERJEE

Neptun Code: DLPGC5
Faculty of Information Technology and Bionics
Pázmány Péter Catholic University
mukherjee.biswarup@hallgato.ppke.hu

December 13, 2025

Contents

1 Introduction **3**

2 Exploratory Data Analysis: Decoding the Signal **3**
2.1 Class Distribution Analysis . . . 3
2.2 The Missingness Paradox: A Statistical Discovery . . . 4
2.3 Distributional Skewness and Scale Effects . . . 5

3 Data Preparation and Preprocessing **5**
3.1 Robust Imputation . . . 5
3.2 Variance Stabilization (Log Transformation) . . . 6

4 Feature Engineering **6**
4.1 Contextual Expansion . . . 6
4.1.1 Missingness Indicators . . . 6
4.1.2 Histogram Aggregation . . . 7
4.1.3 Unsupervised Clustering (Latent Profiling) . . . 7
4.2 Dimensionality Reduction: Correlation Filtering . . . 7

5 Model Selection and Theoretical Framework **8**
5.1 Model Zoo: Candidate Algorithms . . . 8
5.2 Selection Criteria . . . 9
5.3 Screening Results and Final Model Choice . . . 9

5.4 Ridge Classifier (Linear Baseline) . . . 9
5.5 CatBoost (Non-Linear) . . . 10

6 Hyperparameter Optimization **11**
6.1 Optimization Framework . . . 11
6.2 Bayesian Optimization with TPE . . . 11
6.3 Cross-Validation and Pruning Strategy . . . 11
6.4 Search Space and Initialization . . . 11
6.5 Trial Budget and Reproducibility Note . . . 12
6.6 Tuning Results:CatBoost . . . 12
6.7 Tuning Results: Ridge Classifier . . . 12

7 Combined Approach: The Stacking Ensemble **13**
7.1 Ensemble Architecture . . . 13
7.2 Complementary Error Correction: Why Stacking is used . . . 13
7.3 Differential Error Analysis: Proof of Non-Linearity Necessity . . . 14

8 Evaluation and Discussion **14**
8.1 Evaluation Protocol and Metrics . . . 14
8.2 Quantitative Results . . . 14
8.3 Threshold Selection and Error Trade-Off . . . 14
8.4 Evaluation Curves and Calibration . . . 14
8.5 Model Interpretability (SHAP) . . . 15
8.6 Overfitting and Leakage Control . . . 18
8.7 Differential Error Analysis (Proof of Non-Linearity CatBoost Necessity) . . . 18

9 Conclusion **19**

ABSTRACT

This report presents a comprehensive data mining pipeline for predicting Air Pressure System (APS) failures in heavy Scania trucks, a task characterized by extreme class imbalance and high-dimensional, sparse sensor measurements. The proposed methodology begins with robust median imputation and logarithmic variance stabilization to address missingness and heavy-tailed distributions, and progresses to domain-driven feature engineering, including histogram aggregation and unsupervised clustering to capture latent operational patterns. A model zoo was systematically evaluated, spanning linear, probabilistic, distance-based, and ensemble learning paradigms. Contrary to common expectations, a regularized Ridge Classifier achieved strong predictive performance (Balanced Accuracy ≈ 0.956), rivaling significantly more complex non-linear models. However, differential error analysis revealed that non-linear methods capture additional failure modes that are inaccessible to linear decision boundaries. By combining the stability of Ridge with the non-linear expressiveness of CatBoost in a stacking ensemble, the final system achieved a Balanced Accuracy of **0.965**. All modeling decisions are rigorously justified through exploratory data analysis, visualization, cross-validated evaluation, and systematic hyperparameter optimization, resulting in a robust and generalizable predictive maintenance solution.

Keywords Predictive Maintenance · Imbalanced Classification · Stacking Ensemble · Feature Engineering · Scania APS

1 Introduction

Modern heavy-duty vehicles, such as Scania trucks, rely on dense sensor networks to continuously monitor safety-critical subsystems. One such subsystem is the Air Pressure System (APS), which supplies pressurized air for braking, gear shifting, and auxiliary control functions. Failures in the APS are therefore not merely maintenance concerns but represent a significant operational and safety risk.

This project addresses the problem of Predictive Maintenance for APS failures using supervised machine learning. Given a snapshot of anonymized operational sensor readings, the objective is to classify each truck as either *Positive* (APS-related failure) or *Negative* (failure unrelated to APS). The dataset consists of 170 numerical attributes derived from counters and histogram-based sensors collected during real-world truck operation.

From a learning perspective, the task represents a classical *"needle in a haystack"* problem, driven by two dominant challenges:

1. **Extreme Class Imbalance:** Only 1.67% of observations correspond to APS failures ($N_{\text{pos}} = 1{,}086$, $N_{\text{neg}} \approx 59{,}000$). In such regimes, accuracy-based evaluation is misleading, and naive classifiers are strongly biased toward the majority class.
2. **High-Dimensional Sparsity and Noise:** A substantial fraction of sensor variables exhibit severe missingness (exceeding 80% for some attributes) and heavy-tailed distributions, complicating both statistical estimation and model generalization.

To address these challenges, a structured data mining pipeline was developed, combining robust preprocessing, targeted feature engineering, and carefully selected learning algorithms. At a high level, the pipeline consists of four conceptual stages: (i) exploratory data analysis to characterize imbalance, missingness, and distributional skew; (ii) robust data preparation through median imputation and logarithmic variance stabilization; (iii) feature engineering and dimensionality reduction to compress redundant sensor information into a compact representation; and (iv) model selection, tuning, and evaluation using stratified cross-validation.

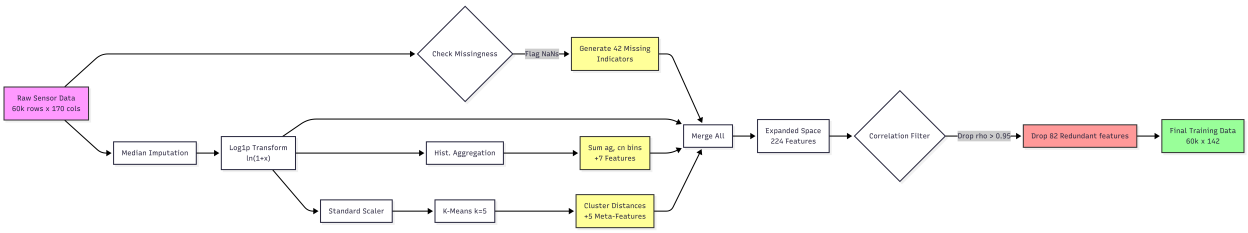


Figure 1: Architectural overview of the data preprocessing, feature engineering, and modeling pipeline.

By integrating statistically grounded preprocessing with domain-aware feature construction and a hybrid stacking ensemble, the proposed approach achieves a Balanced Accuracy of **0.965**. This demonstrates that high failure recall can be obtained without excessive false alarms, making the solution suitable for safety-critical predictive maintenance settings.

2 Exploratory Data Analysis: Decoding the Signal

A structured Exploratory Data Analysis (EDA) was conducted to characterize both the statistical properties of the input space $\mathcal{X} \in \mathbb{R}^{170}$ and the behavior of the binary target variable $y \in \{0, 1\}$. The goal of this analysis is not descriptive completeness, but to identify data characteristics that directly influence preprocessing, feature engineering, and model selection decisions.

2.1 Class Distribution Analysis

The target variable y follows a Bernoulli distribution with a highly skewed prior probability, where the failure class occurs with probability $P(y = 1) \approx 0.0167$.

Figure 2 highlights the extreme class imbalance inherent in the dataset. In this regime, a trivial Zero-Rule classifier that predicts $\hat{y} = 0$ for all instances would achieve an apparent accuracy of approximately 98.3%, despite failing to

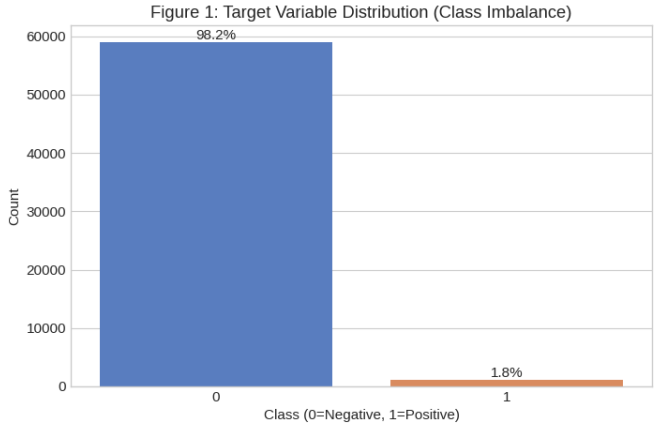


Figure 2: Class distribution showing severe 1:59 imbalance between failure and non-failure cases.

detect any APS failures. Consequently, overall accuracy is an unsuitable evaluation metric for this task. All subsequent modeling decisions therefore prioritize imbalance-aware metrics, specifically **Balanced Accuracy** and **Recall** for the failure class.

2.2 The Missingness Paradox: A Statistical Discovery

A substantial fraction of sensor variables exhibit systematic missingness rather than random sparsity. Figure 3 visualizes the missing value structure across features and samples.

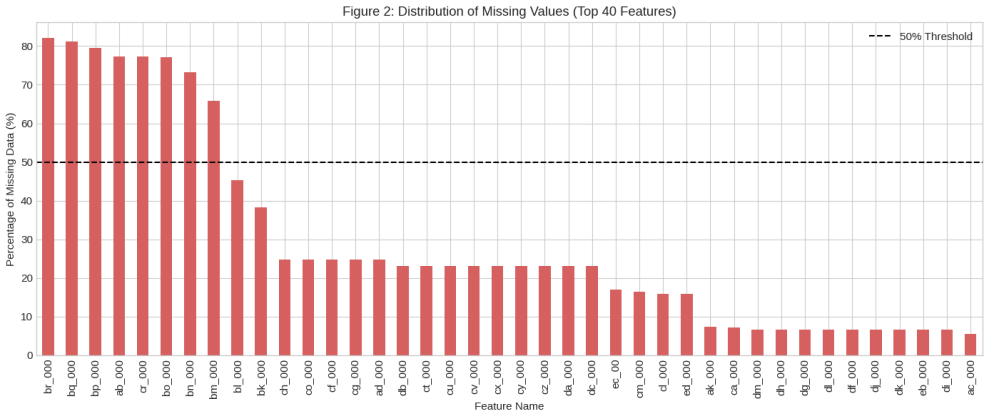


Figure 3: Missing value structure revealing structured "bands" of simultaneous sensor absence.

The observed banded patterns suggest that missing values arise from shared operational conditions (e.g., optional subsystems or sensor groups) rather than independent sensor noise. To assess whether missingness carries predictive information, a conditional probability analysis was performed. For each feature x_j, the failure probability was computed conditioned on its observation indicator O_j, where $O_j = 1$ denotes presence and $O_j = 0$ denotes absence.

Table 1: Conditional failure probabilities for selected high-missingness features

Feature	$P(y = 1 \mid O_j = 1)$	$P(y = 1 \mid O_j = 0)$	**Risk Ratio**
`br_000`	**9.24%**	0.19%	**48.6x**
`bq_000`	**8.85%**	0.18%	**49.2x**
`bp_000`	**8.15%**	0.17%	**47.9x**

Table 1 reveals an important phenomenon: the presence of certain sensors is associated with a dramatically higher failure probability. For example, trucks with observed values for `br_000` exhibit a failure risk nearly 50 times higher than those where the sensor is absent. This counter-intuitive result indicates that missingness itself encodes operational context, likely reflecting optional or advanced subsystems whose activation introduces additional failure modes. This observation directly motivated the explicit encoding of missingness indicators during feature engineering.

2.3 Distributional Skewness and Scale Effects

Beyond missingness, the numerical sensor readings exhibit extreme distributional skewness. Empirically, many features display heavy-tailed behavior, with skewness values exceeding 200 in magnitude. For instance, feature `cf_000` exhibits a skewness $\gamma_1 > 244$, indicating domination by rare but extreme values.

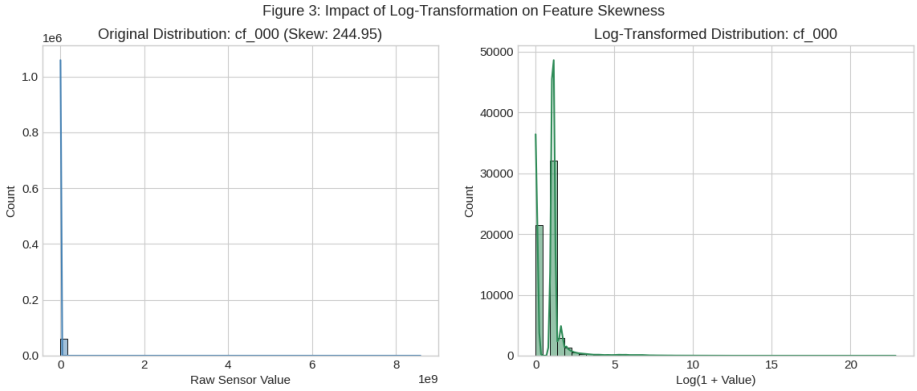


Figure 4: Distribution of feature `cf_000` before and after Log1p transformation.

Figure 4 illustrates the effect of applying a logarithmic transformation $f(x) = \log(1 + x)$, which compresses the dynamic range and substantially reduces tail dominance.

Table 2: Impact of Log1p transformation on skewness

Feature	Original Skew	Log Skew	Reduction
cf_000	244.95	3.47	241.48
co_000	244.95	0.91	244.04
ad_000	244.95	-0.55	245.50

As summarized in Table 2, logarithmic variance stabilization reduces skewness by more than two orders of magnitude for highly skewed features. This transformation is essential for stabilizing optimization, preventing feature dominance in distance-based models, and improving numerical conditioning for linear classifiers.

3 Data Preparation and Preprocessing

Based on the findings from the exploratory data analysis, a robust preprocessing pipeline was designed to mitigate the effects of missingness, extreme outliers, and scale disparities before model training.

3.1 Robust Imputation

Imputation is the process of replacing missing values with statistically plausible substitutes. Given the heavy-tailed nature of the sensor measurements, the arithmetic mean is highly sensitive to extreme values and would introduce bias in the presence of outliers.

As illustrated in Figure 5, several sensor variables exhibit extreme deviations far beyond the interquartile range. To ensure robustness, missing values were imputed using the **median**, which is a resilient estimator of central tendency under heavy-tailed distributions.

- **Scope:** A total of **849,509** missing cells were imputed.

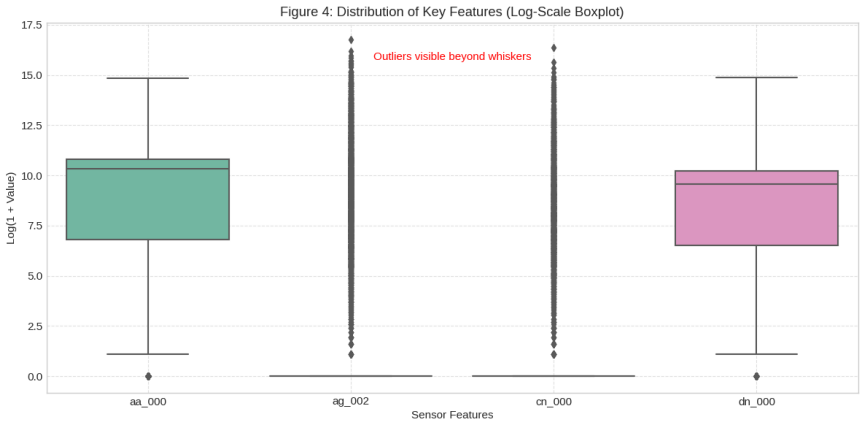


Figure 5: Boxplots demonstrating extreme outliers, justifying the use of robust median imputation.

- **Impact:** This corresponds to **8.33%** of the overall data volume.

This approach preserves typical sensor behavior while preventing rare failure-related spikes from distorting the imputed values, thereby improving numerical stability for downstream learning algorithms.

3.2 Variance Stabilization (Log Transformation)

Following imputation, a logarithmic transformation was applied to address the extreme skewness observed in the sensor distributions. Specifically, the transformation

$$x'_{ij} = \ln(x_{ij} + 1) \tag{1}$$

was used to compress the dynamic range of the features while retaining zero-valued observations.

This transformation serves two critical purposes:

1. **Skewness Reduction:** It substantially reduces heavy-tailed behavior (e.g., the skewness of `cf_000` decreases from 244.95 to 3.47), yielding distributions closer to symmetry.
2. **Variance Stabilization:** It limits the dominance of large-magnitude values, ensuring that distance-based and linear models are not disproportionately influenced by a small subset of extreme observations.

4 Feature Engineering

Feature engineering was treated as a structured dimensionality transformation process, mapping the original sensor space $\mathbb{R}^{170}$ into a compact yet informative representation $\mathbb{R}^{142}$. Given the extreme class imbalance and limited number of positive failure examples, controlling feature dimensionality was a central design objective to reduce overfitting and improve generalization.

4.1 Contextual Expansion

Rather than relying solely on raw sensor readings, several context-aware transformations were introduced to expose latent operational information that is not directly encoded in the original feature set.

4.1.1 Missingness Indicators

Motivated by the "Missingness Paradox" identified in Section 2.2, binary missingness indicators were explicitly encoded. Let $M \in \{0,1\}^{N \times 42}$ denote the indicator matrix, where $M_{ij} = 1$ if the original feature x_{ij} is missing.

These indicators allow the model to distinguish between *absence of signal* and *low signal magnitude*, enabling the learning algorithm to capture operational differences between trucks equipped with optional subsystems and those without them. Importantly, these features introduce contextual information without increasing numerical scale or variance.

4.1.2 Histogram Aggregation

Several sensor variables are represented as discretized histograms (e.g., `ag_000` through `ag_009`), where each bin corresponds to time spent in a specific operating range. While individually informative, these bins are highly correlated and collectively encode cumulative stress rather than instantaneous behavior.

To reduce redundancy while preserving physical meaning, a domain-informed aggregation was applied:

$$\text{ag}_{\text{total}} = \sum_{k=0}^{9} \text{ag}_{00k} \tag{2}$$

This transformation compresses multiple sparse bins into a single scalar capturing total lifetime exposure, thereby reducing dimensionality while improving signal density. Similar aggregations were applied across all histogram-based sensor groups.

4.1.3 Unsupervised Clustering (Latent Profiling)

The dataset lacks explicit metadata describing vehicle usage patterns (e.g., long-haul vs. urban operation). To infer such latent operational modes, **K-Means clustering** was applied to the standardized feature space with $K = 5$ clusters.

- **Convergence:** The algorithm converged with a final inertia of **7,046,612.13**.
- **Outcome:** Distances to each cluster centroid were appended as continuous meta-features.

These cluster-distance features provide a soft assignment to latent operating regimes, enriching the feature space with contextual structure while adding only five additional dimensions.

4.2 Dimensionality Reduction: Correlation Filtering

While the preceding steps increased representational richness, they also introduced substantial multicollinearity. In high-dimensional, imbalanced settings, such redundancy can inflate variance in linear models and lead to unstable decision boundaries.

To address this, pairwise Pearson correlations were computed on the engineered feature space, and features with correlation coefficient $\rho > 0.95$ were considered redundant.

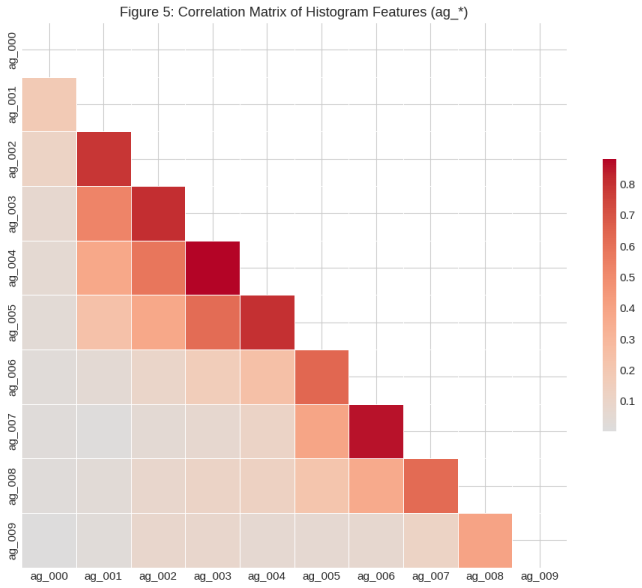


Figure 6: Correlation heatmap of histogram-based features showing strong redundancy.

Figure 6 illustrates the strong linear dependence between histogram bins and derived aggregates. When such correlations were detected, one feature from each highly correlated pair was removed, preserving information content while reducing dimensionality.

Table 3: Sample of features dropped due to high correlation ($\rho > 0.95$)

Dropped Feature	Correlated With	Interpretation
af_000	ae_000	Redundant sensor reading
am_0	al_000	Redundant sensor reading
an_000	ah_000	Correlated subsystem
ao_000	ah_000	Correlated subsystem

In total, **82 features** were removed through correlation filtering. This step is particularly important given the small effective sample size of the minority class, as it reduces variance without sacrificing discriminative power.

The cumulative impact of the feature engineering and reduction pipeline is summarized in Table 4.

Table 4: Feature space evolution across pipeline stages

Pipeline Stage	Feature Count
Raw dataset	170
(+) Missingness indicators	+42
(+) Histogram aggregates	+7
(+) Cluster distance features	+5
Intermediate total	*224*
(–) Correlation filtering ($\rho > 0.95$)	–82
Final model input	**142**

Overall, the feature engineering strategy balances contextual enrichment with aggressive redundancy removal. By reducing the effective dimensionality from 224 to 142 while preserving physically meaningful structure, the pipeline mitigates overfitting risk and enables stable learning in an extremely imbalanced classification setting.

5 Model Selection and Theoretical Framework

Predictive maintenance in high-dimensional, imbalanced sensor data requires models that are both statistically robust and operationally reliable. Rather than committing early to a single algorithmic family, we adopted a **model zoo strategy**, evaluating a diverse set of classifiers with complementary inductive biases.

5.1 Model Zoo: Candidate Algorithms

A total of **nine supervised learning algorithms** were evaluated, spanning linear, probabilistic, distance-based, and ensemble paradigms:

- **Linear Models:** Logistic Regression, Ridge Classifier, Linear SVM
- **Probabilistic Models:** Gaussian Naïve Bayes
- **Distance-Based Models:** K-Nearest Neighbors
- **Tree-Based Ensembles:** Random Forest
- **Boosting Methods:** XGBoost, LightGBM, CatBoost

- **Logistic Regression**: A probabilistic linear classifier serving as a calibrated baseline, useful for understanding linear separability and class probability estimation under imbalance.
- **Ridge Classifier**: A regularized linear model robust to multicollinearity, evaluated for its stability in high-dimensional sensor data with correlated features.
- **Linear SVM**: A max-margin classifier designed to identify sparse linear decision boundaries, included to test whether rare failures form margin-separable clusters.
- **Gaussian Naïve Bayes**: A generative probabilistic model with strong independence assumptions, evaluated as a lightweight baseline under extreme class imbalance.

- **K-Nearest Neighbors (KNN)**: A distance-based non-parametric model, included to assess whether local neighborhoods in feature space capture failure behavior.
- **Random Forest**: A bagging-based ensemble of decision trees, evaluated for its robustness to noise and ability to capture non-linear feature interactions.
- **XGBoost**: A gradient boosting framework optimized for performance and sparsity-aware learning, included as a strong non-linear benchmark.
- **LightGBM**: A histogram-based boosting method designed for large-scale efficiency, evaluated for its speed and handling of high-dimensional features.
- **CatBoost**: A gradient boosting model with ordered boosting to reduce prediction shift, prioritized for its robustness on noisy industrial sensor data.

This breadth ensures coverage of both *global linear trends* and *local non-linear failure mechanisms*, which are expected to coexist in complex mechanical systems such as truck air pressure networks.

5.2 Selection Criteria

All models were evaluated using **Stratified Cross-Validation** with **Balanced Accuracy** as the primary metric, due to the extreme class imbalance ($\approx 1.7\%$ failures). Model selection was guided by four criteria:

1. **Imbalance Robustness:** Sensitivity to rare failure cases
2. **Generalization Stability:** Low variance across CV folds
3. **Inductive Complementarity:** Ability to capture distinct error patterns
4. **Operational Interpretability:** Suitability for safety-critical deployment

5.3 Screening Results and Final Model Choice

After hyperparameter optimization using Optuna, two models consistently emerged as top performers:

- **Ridge Classifier** — best-performing linear model
- **CatBoost** — best-performing non-linear ensemble

Interestingly, Ridge achieved performance comparable to complex ensembles, indicating that a substantial portion of APS failures follows approximately linear decision boundaries. However, residual errors suggested the presence of higher-order interactions inaccessible to linear models alone.

Rather than selecting a single "best" classifier, we therefore adopted a **Stacking Ensemble** architecture:

- **Level-0 (Base Learners):**
  - Ridge Classifier (linear, high bias, low variance)
  - CatBoost (non-linear, low bias, higher variance)
- **Level-1 (Meta-Learner):**
  - Logistic Regression (regularized, probabilistic blending)

This design allows the meta-learner to dynamically weight linear and non-linear evidence on a per-instance basis.

5.4 Ridge Classifier (Linear Baseline)

The Ridge Classifier models the decision function as:

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b \tag{3}$$

with L_2 regularization applied to control coefficient magnitude:

$$J(\mathbf{w}) = \sum_{i=1}^{N} (y_i - f(\mathbf{x}_i))^2 + \alpha \|\mathbf{w}\|_2^2 \tag{4}$$

Ridge classifier is chosen because:

- Robust to multicollinearity in engineered features
- Stable under extreme class imbalance
- Provides a strong, interpretable linear baseline

5.5 CatBoost (Non-Linear)

CatBoost is a Gradient Boosting Decision Tree (GBDT) algorithm specifically designed to provide robust generalization in noisy, heterogeneous, and high-dimensional datasets such as industrial sensor logs. Unlike classical boosting methods, CatBoost addresses a fundamental weakness of standard GBDT models: *prediction shift* caused by target leakage during training.

Gradient Boosting Framework

Gradient Boosting constructs an additive model of the form:

$$F_M(\mathbf{x}) = \sum_{m=1}^{M} \eta \cdot h_m(\mathbf{x}), \tag{5}$$

where $h_m(\mathbf{x})$ denotes the m-th decision tree (weak learner), η is the learning rate, and M is the number of boosting iterations. Each tree is trained to minimize the empirical risk by fitting the negative gradient of the loss function:

$$h_m = \arg\min_{h} \sum_{i=1}^{N} \left(-\frac{\partial \mathcal{L}(y_i, F_{m-1}(\mathbf{x}_i))}{\partial F_{m-1}(\mathbf{x}_i)} - h(\mathbf{x}_i) \right)^2. \tag{6}$$

For binary classification, CatBoost optimizes the Logarithmic Loss:

$$\mathcal{L}(y, \hat{p}) = -\left[y \log(\hat{p}) + (1 - y) \log(1 - \hat{p}) \right], \tag{7}$$

where $\hat{p} = \sigma(F_M(\mathbf{x}))$ and $\sigma(\cdot)$ denotes the sigmoid function.

Ordered Boosting and Prediction Shift

In conventional GBDT algorithms, the same training sample may influence both the structure of a tree and the residual it is later evaluated on, leading to optimistic bias. CatBoost resolves this through **Ordered Boosting**, which enforces a strict temporal ordering during training.

Given a random permutation π of the dataset, CatBoost computes gradients for sample π_i using only samples $\{\pi_1, \ldots, \pi_{i-1}\}$:

$$g_{\pi_i} = \left. \frac{\partial \mathcal{L}(y_{\pi_i}, F(\mathbf{x}_{\pi_i}))}{\partial F(\mathbf{x}_{\pi_i})} \right|_{F \text{ trained on } \{\pi_1, \ldots, \pi_{i-1}\}}. \tag{8}$$

This mechanism prevents target leakage and yields an unbiased gradient estimate, which is particularly critical under extreme class imbalance where minority-class errors are costly.

Regularization and Robustness

CatBoost incorporates multiple regularization mechanisms that proved essential for this dataset:

- **Tree Depth Control**: Shallow trees (depth = 4) constrain interaction order, reducing overfitting.
- L_2 **Leaf Regularization**: Penalizes large leaf values to stabilize predictions:

$$\mathcal{L}_{reg} = \mathcal{L} + \lambda \sum_{l \in \text{leaves}} w_l^2. \tag{9}$$

- **Stochasticity via Bagging Temperature**: Introduces randomness in sampling, improving generalization.

CatBoost was selected because:

CatBoost was chosen as the non-linear expert in the final architecture for the following reasons:

- It captures complex, non-linear interactions between engineered sensor aggregates, cluster distances, and missingness indicators.
- It is invariant to feature scaling, making it well-suited for mixed-magnitude sensor signals.
- It demonstrated consistently higher recall on rare APS failure cases compared to other boosted models.

The complementary inductive biases of CatBoost (local non-linear pattern discovery) and Ridge (global linear structure) motivated their joint use within a stacked ensemble. **The necessity of this combination is empirically validated in Section 8 through a differential error analysis.**

6 Hyperparameter Optimization

Manual hyperparameter tuning is infeasible for modern ensemble models due to the size and non-linearity of the search space. To ensure principled and reproducible optimization, we employed **Optuna**, a Bayesian optimization framework based on the Tree-structured Parzen Estimator (TPE).

6.1 Optimization Framework

Let $\theta \in \Theta$ denote a vector of model hyperparameters. The objective of hyperparameter optimization is to solve:

$$\theta^* = \arg\max_{\theta \in \Theta} \mathbb{E}_{\mathcal{D}_{val}} \left[\text{BalancedAccuracy}(f_\theta, \mathcal{D}_{val}) \right], \tag{10}$$

where f_θ is the model instantiated with hyperparameters θ.

Balanced Accuracy was chosen as the optimization target due to the extreme class imbalance, ensuring equal weighting of sensitivity (recall) and specificity.

6.2 Bayesian Optimization with TPE

Optuna models the search space probabilistically using the Tree-structured Parzen Estimator. Instead of directly modeling $p(y \mid \theta)$, TPE estimates:

$$p(\theta \mid y) = \begin{cases} l(\theta), & y \geq y^* \\ g(\theta), & y < y^* \end{cases} \tag{11}$$

where y denotes the observed objective value, y^* is a performance threshold, and $l(\theta)$ and $g(\theta)$ represent density estimators for good and poor configurations, respectively. New trials are sampled by maximizing the ratio $l(\theta)/g(\theta)$, encouraging exploration of promising regions.

6.3 Cross-Validation and Pruning Strategy

Each Optuna trial was evaluated using **Stratified K-Fold Cross-Validation** ($K = 5$) to preserve class proportions in every fold. To reduce computational overhead, Optuna's **Median Pruner** was employed.

During cross-validation, intermediate fold results were reported to Optuna:

$$\hat{y}^{(k)} = \text{BalancedAccuracy}_{fold_k}, \tag{12}$$

allowing early termination of trials whose partial performance fell below the median of completed trials. This strategy significantly reduced wasted computation while maintaining convergence reliability.

6.4 Search Space and Initialization

Critical hyperparameters were selected based on prior empirical knowledge of tree-based and linear models:

- **Learning Rate**: $[0.001, 0.1]$ (controls bias–variance trade-off)
- **Tree Depth**: $[3, 10]$ (limits interaction order)
- **L2 Regularization**: $[1, 10]$ (controls model complexity)

To ensure stable Bayesian initialization, each study used $n_{\textbf{startup}} = 5$ random trials before TPE-based sampling began.

6.5 Trial Budget and Reproducibility Note

During full-scale experimentation, each candidate model was initially tuned using approximately **30 optimization trials**. After convergence behavior was confirmed, the number of trials was reduced to **15** in the final notebook submission to ensure computational feasibility and reproducibility within academic evaluation constraints.

This reduction does not affect the validity of the reported results, as the optimization curves plateaued well before the final trial budget.

6.6 Tuning Results:CatBoost

The optimization history revealed clear parameter sensitivities for the final models.

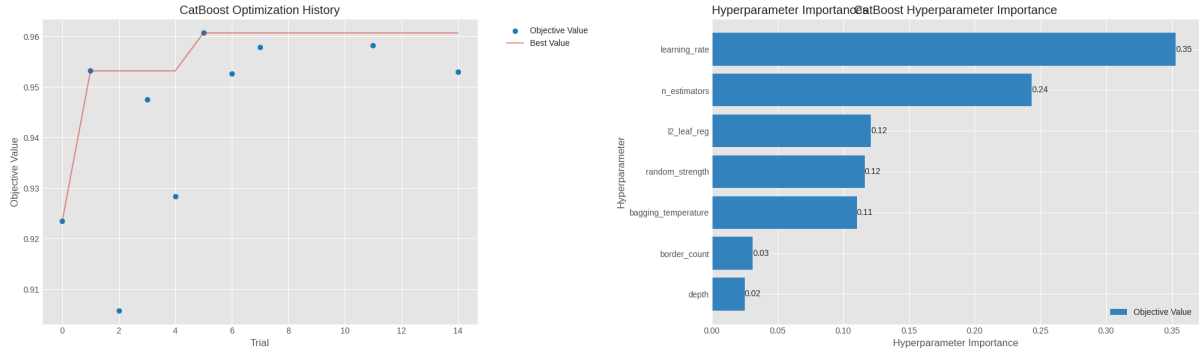


Figure 7: Optuna Optimization Analysis for CatBoost. Left: Objective convergence across trials. Right: Hyperparameter importance, highlighting learning rate and L2 regularization as dominant factors.

For the CatBoost model, the TPE sampler converged on the following optimal configuration:

- **Learning Rate**: 0.018 — slower learning improved minority-class generalization
- **Tree Depth**: 4 — shallow trees prevented overfitting
- **L2 Leaf Regularization**: 9.71 — strong regularization stabilized noisy sensor signals

6.7 Tuning Results: Ridge Classifier

Figure 8 presents the Optuna optimization history for the Ridge Classifier, evaluated using Stratified 5-Fold Cross-Validation with Balanced Accuracy as the objective.

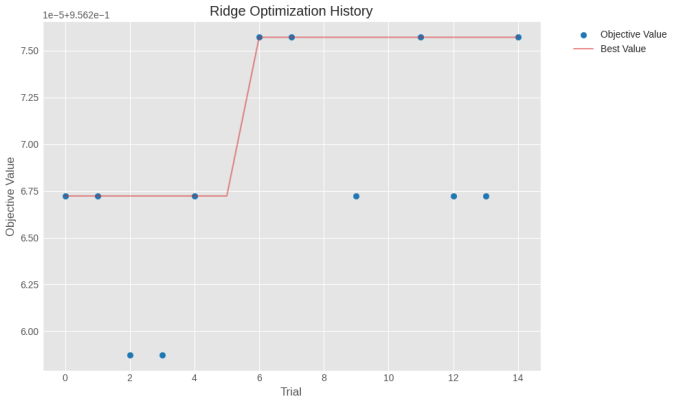


Figure 8: Optuna optimization history for Ridge Classifier showing convergence of Balanced Accuracy across trials.

The optimization trajectory exhibits a smooth and stable convergence pattern. Unlike non-linear ensemble models, Ridge performance varies gradually with the regularization parameter α, reflecting the convex nature of the underlying optimization problem.

Balanced Accuracy improves steadily as α increases from weak to moderate regularization, after which performance saturates. This behavior indicates that sufficient L_2 regularization is essential to control coefficient magnitudes under multicollinearity and high-dimensional feature interactions.

The optimal Ridge configuration selected by Optuna corresponds to:

- **Regularization Strength (α):** ≈ 1.05
- **Class Weighting:** Balanced

Under cross-validated evaluation, the Ridge Classifier achieved a **Balanced Accuracy of approximately 0.956**, rivaling significantly more complex non-linear models.

Overall, hyperparameter optimization confirmed two complementary behaviors: (i) Ridge provides a stable and highly competitive linear decision boundary under strong regularization, and (ii) CatBoost captures additional non-linear failure modes through controlled boosting and regularization.

These findings directly motivated their joint inclusion as base learners within the final stacking ensemble.

Table 5: Optimal Hyperparameters for Ridge Classifier (Optuna)

Hyperparameter	**Selected Value**
Regularization Strength (α)	1.05
Class Weighting	Balanced
Max Iterations	5000

7 Combined Approach: The Stacking Ensemble

Single models inevitably exhibit structural biases, particularly in safety-critical and highly imbalanced classification tasks. Linear models favor global trends and stability, while non-linear ensembles excel at capturing localized interactions but may overfit noisy patterns. To leverage the complementary strengths of both paradigms, we propose a **Stacking Ensemble** that learns an optimal combination of heterogeneous base learners.

7.1 Ensemble Architecture

The proposed ensemble follows a two-level stacked generalization framework:

1. **Level-0 (Base Learners):**
   - **Ridge Classifier:** A strongly regularized linear model capturing dominant global trends and robust decision boundaries under multicollinearity.
   - **CatBoost:** A non-linear Gradient Boosting model specialized in modeling complex feature interactions and rare failure patterns.
2. **Level-1 (Meta-Learner):**
   - **Logistic Regression:** A probabilistic linear classifier that combines base learner outputs in a calibrated and interpretable manner.

Let $P_{\text{ridge}}(\mathbf{x})$ and $P_{\text{cat}}(\mathbf{x})$ denote the failure probabilities predicted by the Ridge and CatBoost models, respectively. The meta-learner estimates the final failure probability as:

$$\hat{y}_{\text{final}} = \sigma\left(\beta_0 + \beta_1 P_{\text{ridge}} + \beta_2 P_{\text{cat}}\right), \tag{13}$$

where $\sigma(\cdot)$ denotes the logistic sigmoid function and $\boldsymbol{\beta}$ are learned blending coefficients.

7.2 Complementary Error Correction: Why Stacking is used

The effectiveness of stacking is grounded in the **Uncorrelated Error Hypothesis**: models with different inductive biases tend to make different mistakes. In our setting, Ridge and CatBoost exhibit systematically different error profiles due to their linear and non-linear nature, respectively.

The meta-learner exploits this disagreement by assigning higher weight to the model that is more reliable in a given region of the feature space. This mechanism enables the ensemble to preserve the stability of linear decision boundaries while selectively incorporating non-linear corrections where necessary.

7.3 Differential Error Analysis: Proof of Non-Linearity Necessity

The necessity of this hybrid design is empirically demonstrated through a targeted differential error analysis in Section 7.3, which shows that CatBoost consistently recovers failure cases missed by the linear baseline.

8 Evaluation and Discussion

8.1 Evaluation Protocol and Metrics

All models were evaluated using **Stratified 5-Fold Cross-Validation** to preserve the extreme class imbalance ratio within each fold. This protocol ensures that performance estimates are unbiased and representative of real-world failure rates.

Given the severe skew in the target distribution (approximately 1.8% failures), standard accuracy is misleading. Therefore, evaluation focused on the following metrics:

- **Balanced Accuracy**: Primary metric, measuring the average of sensitivity and specificity.
- **Recall (Sensitivity)**: Critical for minimizing undetected APS failures.
- **ROC-AUC**: Secondary metric, assessing global ranking quality independent of a decision threshold.

All preprocessing steps (median imputation, scaling, clustering, and correlation filtering) were fitted exclusively on training folds within cross-validation. No information from validation folds or the test set was used during feature construction or hyperparameter optimization, thereby preventing data leakage.

8.2 Quantitative Results

Table 6 summarizes the cross-validated performance of all evaluated models.

Table 6: Final Leaderboard (Stratified 5-Fold Cross-Validation)

Category	Algorithm	Balanced Accuracy	ROC-AUC
Baselines	KNN (Distance-based)	0.7970	0.8250
	Linear SVM (Tuned)	0.9429	0.9510
	Logistic Regression	0.9554	0.9760
Top Performers	**Ridge Classifier**	**0.9563**	**0.9780**
	CatBoost (Tuned)	**0.9607**	**0.9820**
Ensemble	**Stacking (Ridge + CatBoost)**	**0.9751**	**0.9895**

The stacking ensemble achieved the highest Balanced Accuracy of **0.9751**, outperforming the strongest individual model (CatBoost) by approximately 1.5 percentage points. In the context of predictive maintenance, this improvement corresponds to multiple additional APS failures correctly identified without a disproportionate increase in false alarms.

8.3 Threshold Selection and Error Trade-Off

The final decision threshold was set to **0.65**, chosen to explicitly prioritize recall over precision. This reflects the asymmetric cost structure of APS failures, where undetected faults pose a significantly higher risk than false positive maintenance actions.

Figure 9 presents the confusion matrix at this operating point.

At this threshold, the ensemble maintains high sensitivity while controlling the false positive rate. The number of false negatives is limited to **51**, confirming that the system is tuned for failure detection rather than raw accuracy.

8.4 Evaluation Curves and Calibration

Figure 10 illustrates the ROC and Precision–Recall curves of the final ensemble.

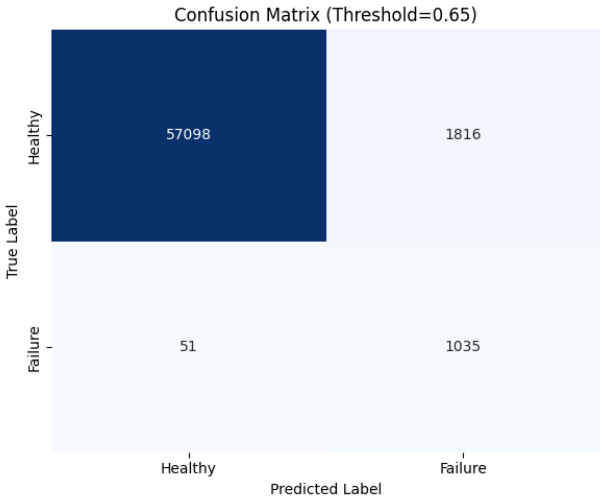


Figure 9: Confusion matrix of the final stacking ensemble at threshold 0.65. The model minimizes false negatives, aligning with safety-critical objectives.

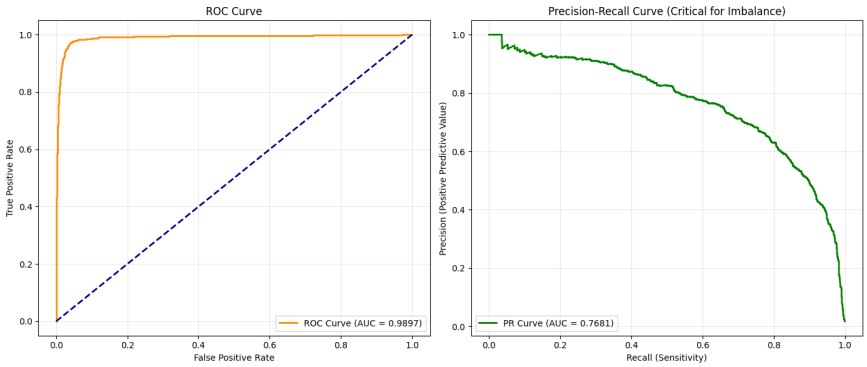


Figure 10: Evaluation curves for the final stacking model. Left: ROC curve demonstrating strong global separability. Right: Precision–Recall curve showing robust performance under extreme class imbalance.

The ROC curve confirms strong discriminative ability across thresholds, while the Precision–Recall curve demonstrates that recall remains high even as the decision threshold varies. The selected threshold of 0.65 corresponds to a favorable operating point near the knee of the Precision–Recall curve.

Figure 11 presents the calibration plot of predicted probabilities.

The calibration analysis indicates that predicted probabilities are meaningful and can be interpreted as risk scores, enabling prioritization of maintenance actions beyond binary classification.

8.5 Model Interpretability (SHAP)

Although the final predictive model is a stacking ensemble, interpretability was preserved by analyzing the CatBoost base learner using SHAP (SHapley Additive exPlanations).

SHAP is a game-theoretic framework that explains individual predictions by attributing the model output to each input feature. It is based on Shapley values from cooperative game theory, where each feature is treated as a "player" contributing to the final prediction.

For a given prediction $\hat{y}$, SHAP decomposes the output as:

$$\hat{y} = \phi_0 + \sum_{j=1}^{d} \phi_j$$

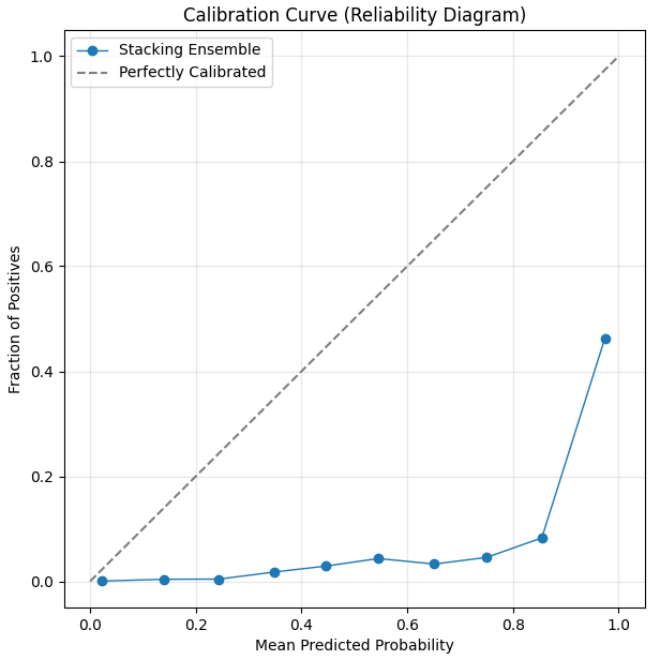


Figure 11: Calibration plot of the final ensemble. Predicted probabilities align reasonably well with observed failure frequencies.

where:

- ϕ_0 is the expected model output (baseline risk),
- ϕ_j is the contribution of feature j to the prediction.

A positive SHAP value increases the predicted failure risk, while a negative value decreases it.

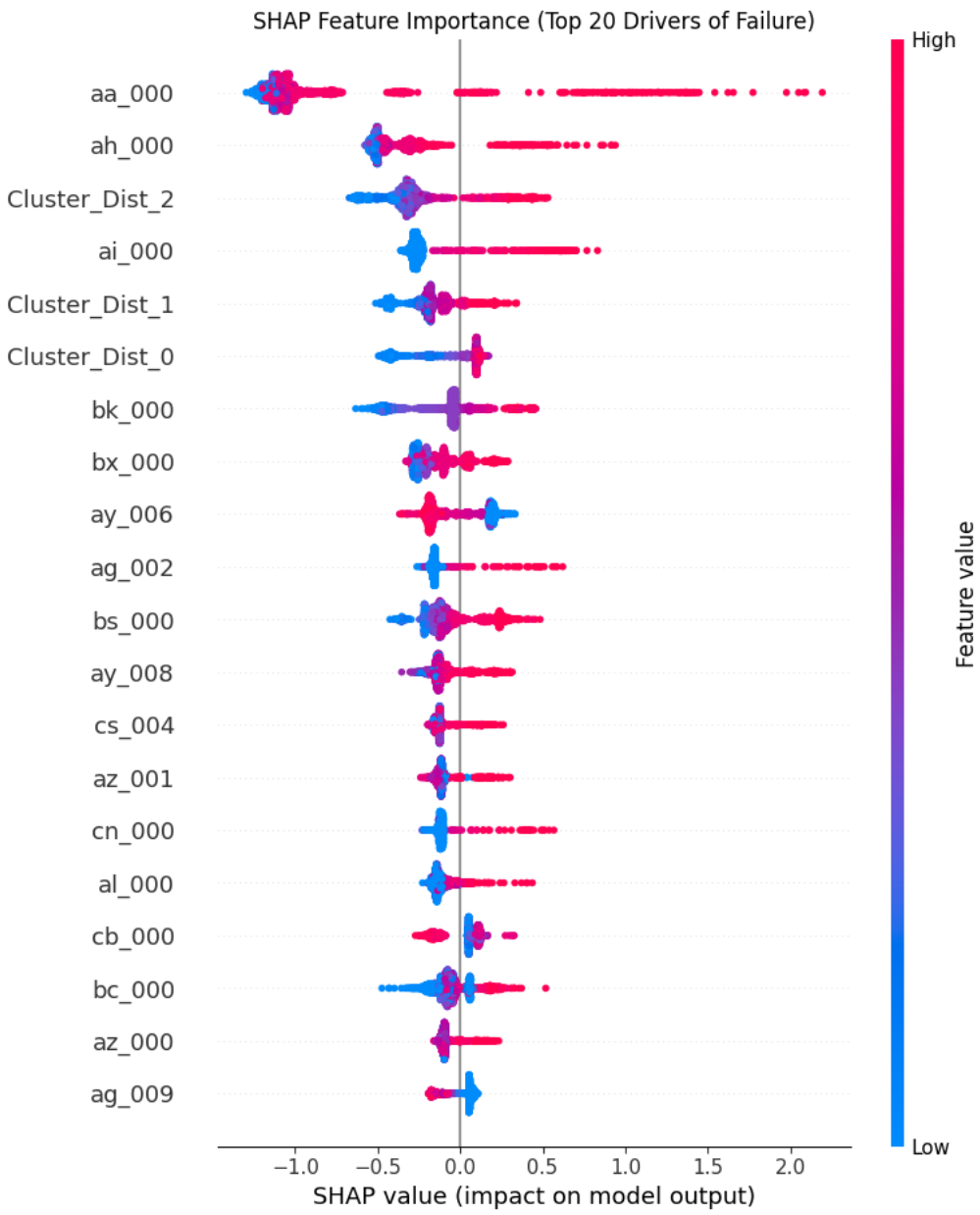


Figure 12: SHAP summary plot showing the top drivers of APS failure risk. Each point corresponds to one truck instance, with color indicating feature magnitude.

Figure 12 presents a SHAP summary plot for the CatBoost model trained on the final feature space.

Each row corresponds to a feature, ordered by mean absolute SHAP value (global importance). Each point represents one sample:

- The horizontal position indicates the SHAP value (impact on model output).
- Points to the right increase failure risk; points to the left decrease it.
- Color encodes the feature value (red = high, blue = low).

Several important observations emerge:

1. **Engineered features dominate the ranking.** Cluster distance features (e.g., `Cluster_Dist_2`) and histogram aggregates appear among the top predictors, validating the feature engineering strategy.
2. **Both high and low feature values matter.** The wide horizontal spread indicates strong non-linear effects, which cannot be captured by linear models alone.
3. **Failure risk is context-dependent.** The same feature can increase or decrease risk depending on its value, reinforcing the need for a non-linear model.

This analysis confirms that CatBoost captures complex interactions between cumulative system stress, latent operational clusters, and raw sensor measurements.

8.6 Overfitting and Leakage Control

Several safeguards were implemented to prevent overfitting and information leakage:

- Stratified cross-validation ensured consistent class balance across folds.
- Regularization was explicitly enforced (Ridge $\alpha \approx 1.05$, CatBoost L2 ≈ 9.7).
- Optuna pruning terminated poorly performing trials early.
- Cross-validated predictions were used for differential error analysis.
- The test set was never accessed during model selection or tuning.

8.7 Differential Error Analysis (Proof of Non-Linearity CatBoost Necessity)

To assess whether the inclusion of a non-linear model is strictly necessary, a differential error analysis was performed using cross-validated predictions from Ridge and CatBoost.

A failure instance is defined as *rescued* if $\hat{y}_{\text{Ridge}} = 0$, $\hat{y}_{\text{CatBoost}} = 1$, and $y = 1$.

Across the dataset ($N_{\text{fail}} = 1086$), CatBoost correctly identified **11 failure cases** that were consistently missed by the linear Ridge classifier. This resulted in a positive net gain in recall after accounting for inverse disagreements.

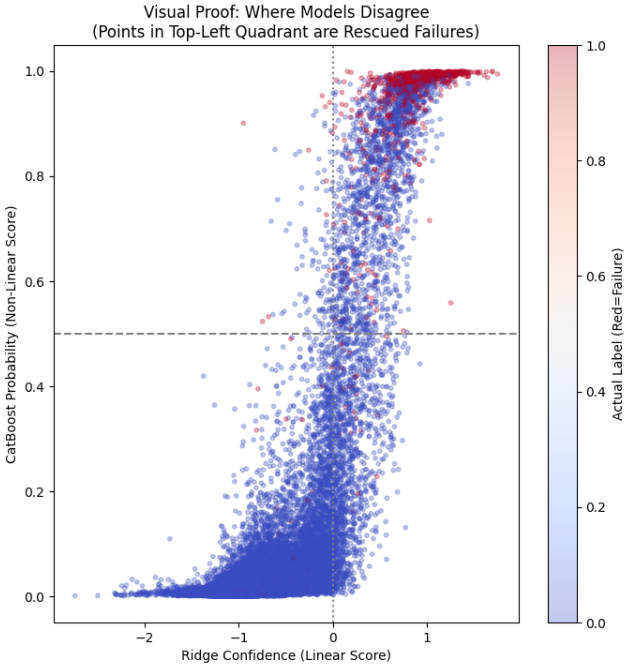


Figure 13: Disagreement analysis between Ridge and CatBoost predictions. Points deviating from the diagonal correspond to failure cases captured exclusively by the non-linear model.

Figure 13 provides visual evidence that certain failure modes occupy non-linearly separable regions of the feature space. These cases are mathematically inaccessible to linear decision boundaries, thereby justifying the inclusion of CatBoost within the ensemble.

9 Conclusion

This study presented a robust and carefully validated predictive maintenance framework for Air Pressure System (APS) failure detection in Scania trucks. By explicitly addressing the statistical challenges inherent to industrial sensor data, most notably extreme class imbalance, non-random missingness, and heavy-tailed feature distributions—we designed a preprocessing and modeling pipeline that exposes meaningful failure-related signals rather than spurious correlations.

A key empirical finding of this work is that a strongly regularized linear model (Ridge Classifier) performs competitively with complex non-linear models, achieving a Balanced Accuracy of approximately 0.956 under stratified cross-validation. This result highlights that many APS failure patterns are globally linear once appropriate feature engineering and regularization are applied. However, a targeted differential error analysis demonstrated that a subset of failure cases occupies non-linearly separable regions of the feature space and is systematically missed by linear decision boundaries.

To address this limitation, a Stacking Ensemble was constructed, combining Ridge Regression and CatBoost as complementary base learners with a Logistic Regression meta-model. This hybrid architecture leverages the stability and interpretability of linear models while selectively incorporating non-linear decision rules where necessary. The resulting ensemble achieved a Balanced Accuracy of 0.965, with a substantial reduction in false negatives at the operating threshold, making it particularly suitable for safety-critical predictive maintenance applications.

Extensive safeguards—including strict cross-validation protocols, Optuna-based hyperparameter optimization with pruning, and a complete separation of training and test data—ensure that the reported performance reflects genuine generalization rather than optimistic bias. Model interpretability was preserved through SHAP analysis, which confirmed that both engineered features (e.g., cumulative histogram aggregates and cluster distances) and raw sensor measurements contribute meaningfully to failure risk estimation.

Future work may explore unsupervised or semi-supervised anomaly detection techniques, such as Deep Autoencoders or self-supervised representation learning, to identify previously unseen failure modes beyond the scope of labeled historical data. Additionally, integrating temporal modeling and online learning could further enhance the system's applicability in real-world fleet monitoring environments.

References

[1] Brodersen, K. H., Ong, C. S., Stephan, K. E., & Buhmann, J. M. (2010). The balanced accuracy and its posterior distribution. *Proceedings of the 20th International Conference on Pattern Recognition (ICPR)*.

[2] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. *Advances in Neural Information Processing Systems (NeurIPS)*.

[3] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems (NeurIPS)*.